Intelligent Association

Presented by

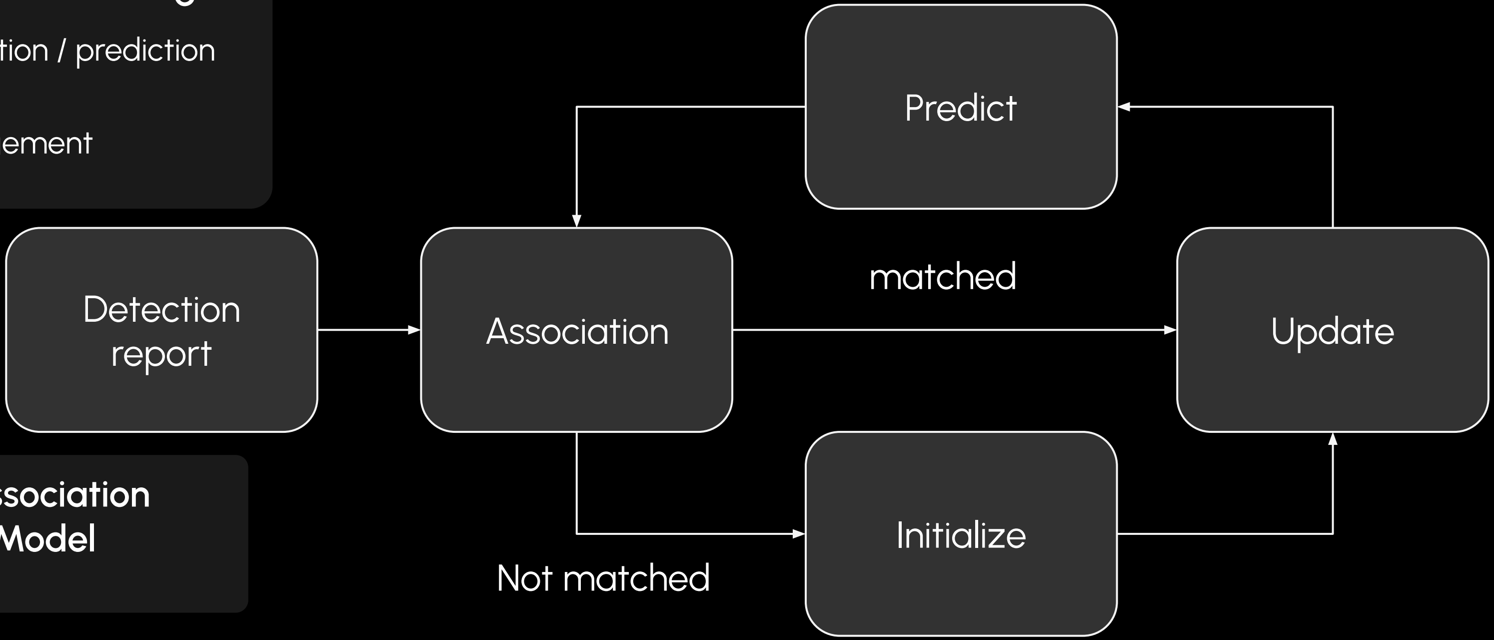
Spencer

Randolph

Background - Tracking

Three main parts of tracking:

- State estimation / prediction
- Association
- Track Management



GOAL: Improve Association Through Learned Model

Milestone 1:

Data Generation

Milestone 2:

Tracking Architecture

Model Training

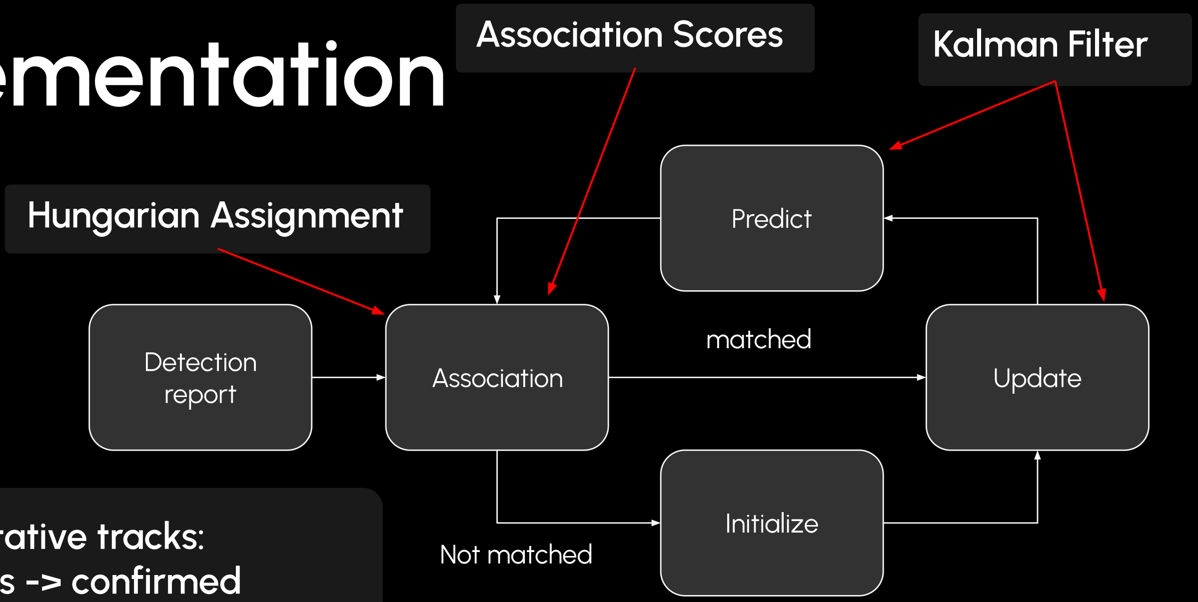
Milestone 3:

Model Training

Evaluation

Tracker implementation

Python
Predict Associate Update Loop
State estimation:
- Kalman Filters



Track Management:
Tentative tracks
Confirmed tracks
Background tracks

Tentative tracks:
3 hits -> confirmed
2 misses -> dropped

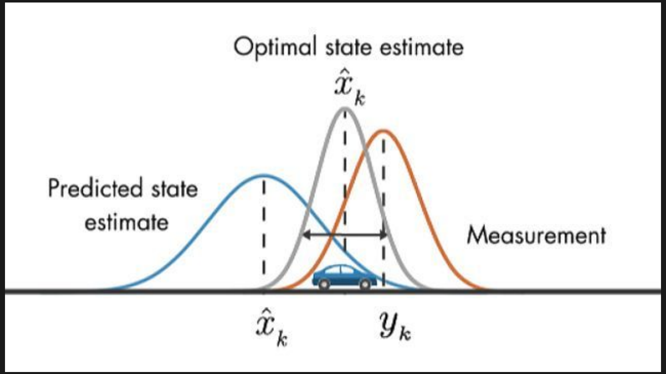
Background tracks:
Alive for next 10 frames
for posible reassociation

Confirmed tracks:
6 misses -> background

Kalman filters

- Recursive state estimation
- predict update cycles
- linear gaussian model

Uncertainty:
- State uncertainty (P)
- Process uncertainty (Q)
- Measurement uncertainty (R)



Predict:
- project the track using estimated velocity
- increases state uncertainty by Q

Update:
- **residual:** difference between predicted , actual measurement
- **kalman gain:** ratio ≈ P / (P + R)
- **state correction**: predicted measurement + kalman gain * residual
- **IMPORTANT**: The **kalman gain updates Pos & Vel**
- **update state uncertainty: p ≈ (1-k)*p**

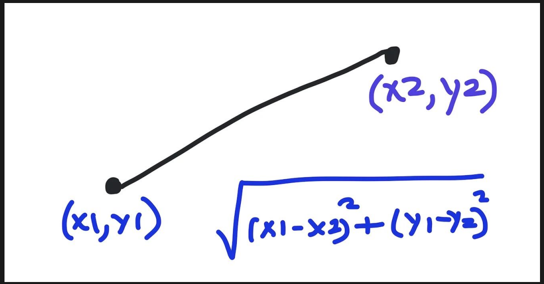
Current Associations

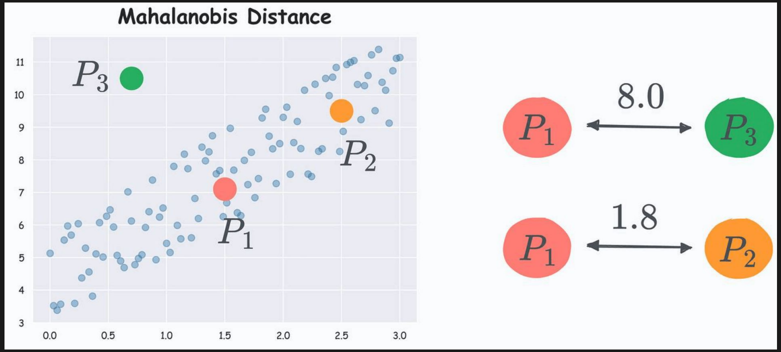
Euclidean Distance:

Straight line distance between prediction and detection

independent of sample size

Simple easy to interpret





Mahalanobis Distance:

Cost ≈ (x - μ) Σ ...

normally:
X: val
μ: mean
Σ: covariance

KF Tracking:
X: detection
μ: prediction
Σ: P and R

Dependent of sample size

Hungarian Assignment

Translates pairwise costs to final 1 to 1 assignment
Time complexity: $O(n^3)$
Where n = max(track, detection)

- Importance of Gating
- Probability is just cost 0-1

Association Cost Matrix

	D1	D2	D3	D4	D5	D6	D7
Track A	1.2	3.8	∞	∞	∞	∞	∞
Track B	∞	∞	2.0	4.1	∞	∞	∞
Track C	∞	∞	∞	∞	1.5	6.3	∞

green = selected match   yellow = valid candidate   gray = gated out

Rows = tracks, columns = detections, lower cost = better match.

Trained model

Drops in place for association

Intelligently Combine:

- associations
- track maturity

Single probability

MLP(PyTorch):

- Scores each candidate track-detection pair
- Input -> hidden layers -> output logit
- ReLU activations
- Sigmoid converts the output into P(match)
- Tracker cost = 1 - P(match)

thoughts:

- Feature set exploration
- Minimal model sizes e.g [32,16] , [64,32], [128,64, 32]

Model training

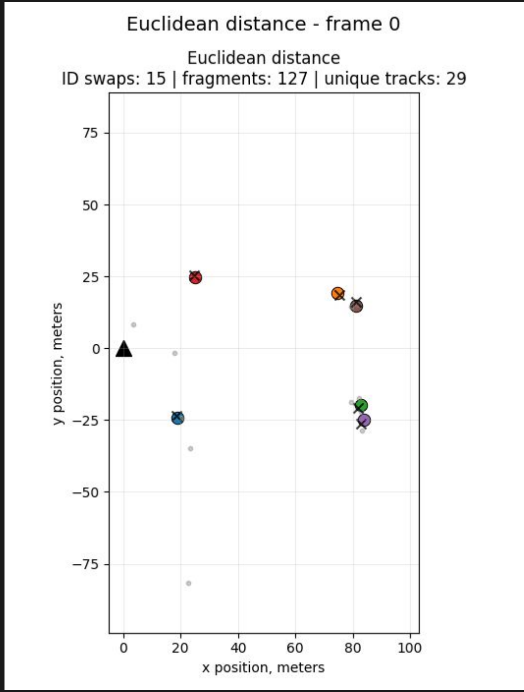
Pair building:
- Iterate through detection report
- Always associate with right detection
- Build paid of (t,c,d,d,d) or (p,d,d,d,d)

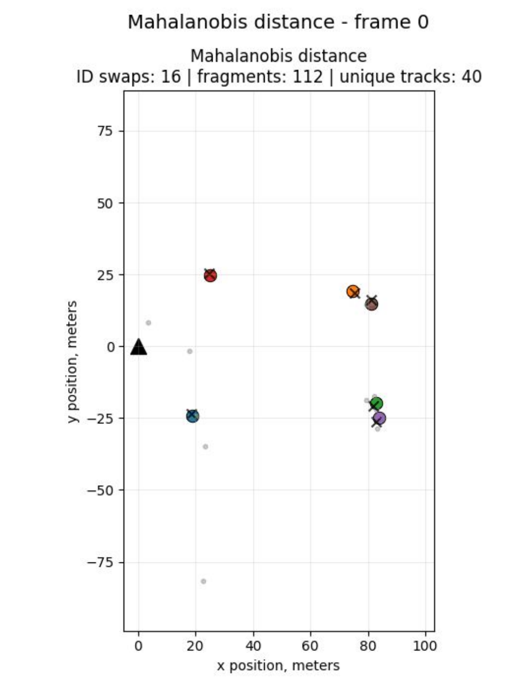
Randomize Data
Split data:
25 % test
75 % training

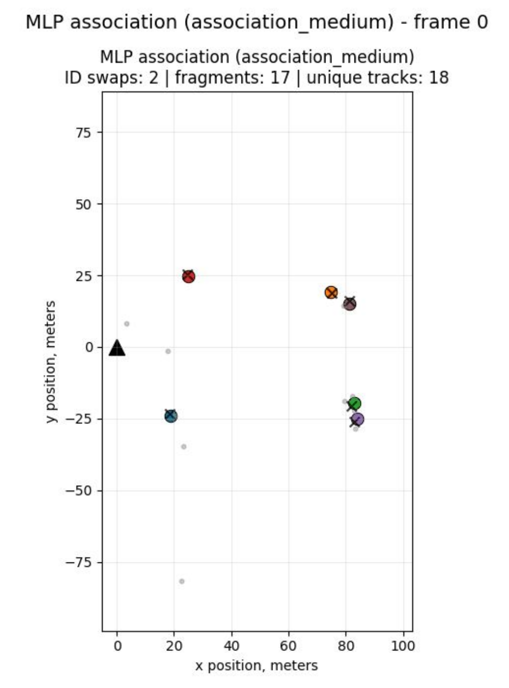
Training:
- Batch Training
- Binary Cross Entropy with Logits

Ran into Issues:
- Highly negative data distribution
- Random sampling for pair building
- Lack of learning
- Feature set creep

Current Early Results (disclaimer untuned gate)







Planned work

Tracking / model:
further training and tuning

further state estimation exploration:
- adaptive kalman filter

Further association / feature set:
- Rcs likelihood model
- Motion explainability model

bug fixing

Evaluation:
- Compare to JPDA, PDA ?
- Test generalisability
- Model metrics ~ f1

Questions?